Commandes de bases, navigation dans les fichiers

`ls`

Liste les fichiers d'un dossier. Options : `-a` pour les fichiers cachés, `-l` pour la liste détaillées, `-h` pour les tailles en unités "human readable". Très pratique l'option `-R` permet en un coup d'œil de visualiser les sous-dossiers.

`cd`

Change directory, la commande permet de naviguer dans l'arborescence. Par exemple `cd /var/log` va dans le dossier des logs, quel que soit l'endroit où l'on se trouve puisqu'on a mis le slash de début, lequel indique qu'il s'agit d'une adresse absolue. En revanche, `cd mondossier/images` va dans le répertoire `images` de `mondossier` lequel se trouve à l'endroit où on se situe déjà. Comme on ne met pas de slash de début, il s'agit d'une adresse relative, on ajoute donc ce chemin à celui dans lequel on se trouve déjà.

Deux raccourcis très utiles sont à connaître. `cd ~` mène au répertoire de l'utilisateur courant (`/home/user/` la plupart du temps ou `/root/` si vous êtes en root) et `cd -` retourne sur le chemin précédent.

`du`

disk usage, précise l'espace disque que prend chaque fichier ou dossier (l'option `-h` permet d'obtenir les tailles en "human readable"), tandis que l'option `--max-depth=x` (ou `-d`), très utile également, permet de limiter la détail à un niveau de sous dossier. `--max-depth=1` ne retournera donc que la taille des répertoires courants. Cette commande s'avère particulièrement pratique quand `ls -l` ne nous donne pas la taille d'un dossier.

`pwd`

print working directory. Cette commande affiche tout simplement le chemin absolu du dossier dans lequel on se trouve,

```
Buzut$ pwd
/etc/apache2
```

`clear`

Nettoie votre fenêtre de terminal en reléguant tout le texte au-dessus (donc accessible avec un scroll) et vous laissant donc face à une fenêtre clean. Bien utile de temps à autre pour y voir plus clair. Le raccourci clavier `ctrl` + `l` fait la même chose.

`Ctrl` + `s`

Stoppe l'affichage, très pratique lorsqu'une commande très verbeuse "pollue" un peu votre terminal. Inversement, pour reprendre l'affichage, on utilisera `ctrl` + `q`

`Ctrl` + `d`

Déconnecte proprement une session ou un terminal. Par exemple, si vous êtes connecté en root depuis sudo, retourne à la session de l'utilisateur qui a exécuté sudo (comme si vous aviez entré `exit`). Ferme la connexion ssh si vous vous êtes connecté sur la session courante. Ce raccourci permet aussi de fermer un interpréteur comme la console MySQL par exemple. Pour être tout à fait exact, cette commande envoie le caractère *EOF* (d'end of file).

`Ctrl` + `k`

Supprime tout le texte se trouvant après le curseur et le sauvegarde dans le presse papier. `Ctrl` + `u` supprime quant à lui depuis le curseur jusqu'au début de la ligne.

`Ctrl` + `y`

Permet de coller le texte présent dans la presse papier.

`Ctrl` + `r`

Très, très pratique, permet de faire une recherche dans l'historique des commandes. Habituellement, vous remontez dans les commandes déjà tapées avec la flèche du haut, eh bien avec `ctrl` + `r`, vous pouvez effectuer une recherche dans cet historique, faites `ctrl` + `r`, puis tapez un bout de la commande que vous voulez rechercher, magique !

`history`

L'historique de bash est très pratique. Cependant, il peut parfois s'avérer intéressant d'en supprimer des entrées. `history` permet d'afficher l'historique avec les numéros de lignes correspondants. Si vous voulez supprimer une entrée, il suffira d'utiliser l'option `-d`, `history -d 1125` et pour tout supprimer, vous utiliserez l'option `-c` sans autre paramètre.

`!!`

Dans la lignée des raccourcis bash bien pratiques, le double point d'exclamation, permet de lancer à nouveau la dernière commande. Ainsi, exécuté après un `ls` ou un `pwd` ré-exécutera cette dernière.

`for`

C'est certainement la commande la plus complexe de cette section, surtout si vous ne programmez pas. `for` est une instruction de boucle. Une boucle permet d'exécuter une action plusieurs fois, sur tous les éléments d'une variable. Par exemple, nous pouvons ainsi très facilement renommer tous les fichiers d'un répertoire pour remplacer les espaces par des traits d'union.

```
# Pour tous les éléments dans le répertoire courant
# L'instruction in place les éléments correspondants dans la variable
oldname
```

```
      for oldname in *
      # do execute l'action en boucle
      do
        # On utilise ici sed pour renommer,
        # l'usage de la commande est expliqué plus bas dans l'article
        newname=`echo $oldname | sed -e 's/ /_/g'`

        # On invoque la commande mv pour remplacer l'ancien nom par le
      nouveau
        mv "$oldname" "$newname"
      done
watch
```

Cette commande permet d'appeler une autre commande de manière répétitive. On peut par exemple vouloir suivre l'évolution du contenu d'un répertoire, `watch ls` permet cela. L'option `-n` permet de spécifier un intervalle en seconde.

Opération sur les fichiers

`cat`

Lire le contenu d'un fichier texte `cat monfichier.txt`

`less`

Fonctionnement similaire à `cat` mais affiche le fichier page par page. C'est donc plus pratique pour les longs fichiers.

`head`

Affiche l'en-tête d'un fichier, l'option `-n` permet de spécifier le nombre de lignes à **afficher.**

`tail`

Semblable à `head` mais concerne la "queue" du fichier, en d'autres termes, cette commande n'affiche que la fin. Une option très appréciable `-f` pour follow, permet de mettre à jour en temps réel l'affichage de la fin du fichier, ce qui est fort pratique pour suivre l'évolution d'un fichier de logs par exemple ;)

`touch`

Créer un fichier. Il suffit de faire `touch nom_du_fichier`. Comme Franckito le précise dans les commentaires `touch` a pour but premier de modifier l'horodatage d'un fichier. Si vous faites `touch` sur un fichier qui existe déjà, cela actualisera ses dates de dernier accès et modification.

`mkdir`

Créer un dossier, le fonctionnement est le même que celui de la commande `touch`.
`mkdir nom_du_dossier`

`cp`

Copy, faire une copie d'un fichier. L'option `-R` permet de réaliser des copies de dossiers entiers.

```
cp fichier_original copie_du_fichier

# on peut aussi placer la copie directement dans un autre dossier
cp fichier_original nom_du_dossier/copie_du_fichier
```

`mv`

Move, permet de déplacer des fichiers/dossiers. La commande mv s'utilise exactement de la même manière que la commande `cp`. En outre, cette commande permet aussi de renommer les fichiers et dossiers en changeant simplement leur nom.

```
mv mon_fichier mon_fichier_new_name
```

`rm`

Remove, supprime des fichiers. `rm nom_du_fichier`. L'option `-f` force la suppression, l'option `-i` demande une confirmation avant suppression, enfin l'option `-r` permet la suppression des dossiers.

`rmdir`

remove directory, supprime un dossier seulement s'il est vide.

`ln`

link, créer un lien entre deux fichiers. L'option `-s` permet de créer un lien symbolique.

```
#créer un lien en dur
ln fichier1 fichier2

#créer un lien symbolique
ln -s fichier1 lien_vers_fichier1
```

`wc`

Word count, permet de compter le nombre de lignes, de mots et de caractères dans un fichier texte. Les options sont `-l` pour line (nombre de ligne), `-w` pour word (nombre de mots) et `-m` pour le nombre de lettres. Il y a aussi l'option `-c` pour avoir la taille du fichier en bits. Pour l'utiliser, on fournit simplement en paramètre l'adresse du fichier texte :

```
Buzut$ wc -l test.rtf
      33 test.rtf
```

`wc` permet aussi facilement de savoir combien vous avez de fichiers/dossiers un dans répertoire donné, il suffit pour cela de rediriger la sortie d'un `ls` vers `wc` : `ls | wc` et le tour est joué !

`sort`

Trier un fichier texte par ordre alphabétique. L'option `-r` permet d'effectuer un tri inverse, c'est à dire anti-alphabétique ou décroissant pour les nombres, et l'option `-R` permet un tri aléatoire, c'est le mode shuffle quoi ;) On n'oubliera pas non plus l'option `-u` qui permet d'éliminer les doublons. Enfin l'option `-o` permet de créer un nouveau fichier avec les résultats triés :

```
sort -o fichier_trie.txt fichier_en_bordel.txt
```

`uniq`

La commande `uniq` permet de dédoublonner un fichier. Il suffit de lui passer en paramètre l'adresse du fichier à dédoublonner et le nom du nouveau fichier à créer.

```
uniq doublons.txt no-doublons.txt
```

`cut`

Couper dans un fichier texte. Pour couper toutes les lignes selon un nombre donnés de caractères, on utilisera l'option `-c`. `cut -c 2` conservera seulement les deux premiers caractères. On peut aussi donner un intervalle : `cut -c 2-4`, alors on conservera uniquement les caractères deux à quatre. Exemple, "anticonstitutionnellement" sera transformé en "ntic". Il est aussi possible de se servir de délimiteurs pour couper du texte, avec les options `-d` et `-f`. Les fichiers au formats .csv séparent les différents champs, les colonnes, par des point-virgules `;`. Dans un fichier où nous aurions trois champs, le nom, le prénom et la ville, si nous voulons extraire la ville, nous ferions comme ceci `cut -d ; -f 3` on indique le délimiteur après `-d` et le champ après `-f` (field veut dire champ en anglais).

`tar`

`tar` est l'utilitaire d'archivage. Il permet de regrouper des fichiers et des dossiers dans une seule archive. Les options intéressantes sont les suivantes : `tar -cvf` (create, verbose, file) permet de créer une archive, d'afficher tous les détails du processus (mode verbeux) et de tout mettre dans un dossier. Exemple :

```
tar -cvf nouvelle_archive.tar mon_dossier_a_archiver
```

Processus inverse, pour "détarrer" une archive, on utilise les options `-xvf` (eXtract, verbose, file) `tar -xvf archive.tar`. Les options `-tf` servent à afficher le contenu d'une archive sans l'ouvrir. Il est aussi possible de compresser et décompresser à la volée les archives `tar`, il faut rajouter pour cela l'option `-z` lors de la création ou l'ouverture de l'archive `tar -zcvf compress.tar.gz compress/`.

`gzip`

Permet de compresser une archive tar au format zip. `gunzip archive.tar.gz`, il suffit ensuite d'utiliser la commande `gunzip` pour la dézipper.

`bzip2`

Fonctionne exactement de la même manière que `gzip` mais compresse au format `bzip`. Pour décompresser l'archive, l'équivalent de `gunzip` est ici `bunzip2`.

`zcat zmore zless`

`zcat`, `zmore` et `zless` remplissent les mêmes fonctions que `cat`, `more` et `less` mais à appliquer aux fichiers compressés.

`iconv`

Permet de changer l'encodage d'un fichier. Option `-f` pour préciser l'encodage d'origine et option `-t` pour celui de destination. Par défaut, `iconv` renvoit tout sur la sortie standard, donc si vous voulez directement envoyer les résultats dans un fichier, il suffit de faire une petite redirection :

```
iconv -f UTF-8 -t UTF-17 fichier.txt >> new-encodage.txt
```

`rsync`

C'est un utilitaire qui permet de synchroniser entre eux des dossiers. Très pratique donc pour la sauvegarde. C'est pour ma part les options `-arv` que j'utilise. `-a` conserve les droits etc, `-r` permet la récursivité et `-v` pour le mode verbeux. Petit exemple de sauvegarde de mes photos de vacances :

```
rsync -arv photo backup_photo
```

Il est utile de préciser que si vous supprimez des fichiers dans le dossier source, `rsync` ne répercute pas la suppression dans le dossier de sauvegarde si vous ne lui adjoignez pas l'option `--delete`. Au cas où vous ne désiriez pas supprimer totalement les fichiers, il est possible de les placer dans un dossier séparé, options : `--backup --backup-dir=` Petit exemple pour la forme ?

```
rsync -arv photo ~/backup_photo --delete --backup --backup-dir=~/backup_photo/delete
```

Bien entendu, il est possible de faire une sauvegarde distante.

```
rsync -arv photo buzut@monserver:~/backup_photo --delete
```

Dans le cas d'une sauvegarde distante, il peut s'avérer très avantageux d'activer la compression, ainsi, `rsync` compresse les fichiers de manière transparente pendant le transfert. Cette option magique a pour petit nom `-z` comme zip.

Dernière chose, au cas où votre serveur n'écoute pas sur le port 22 en ssh, option `-e`, exemple pour un ssh sur le port 443 `-e 'ssh -p 443'`.

`grep`

Permet d'effectuer des recherches par expressions régulières. Dans sa forme la plus simple, `grep` permettra d'afficher la ligne contenant un mot clef (avec l'option `-o` on affiche seulement l'expression matchée), ceci depuis un fichier ou une autre commande.

Par exemple, si on veut afficher tous les processus ssh, on filtrera la commande `ps aux` avec `grep` : `ps aux | grep ssh`. `grep` permet également de rechercher dans le

contenu de fichiers. Un exemple tiré de mon article sur la désinfection d'un WordPress où l'on cherche tout fichier `.php` potentiellement infecté :

```
# option -l pour matcher seulement certains types de fichiers (ici
les .php)
# option -r pour la récursivité (recherche dans les sous-dossiers)
# option -E pour la syntaxe REGEXP étendue (correspond à la commande
egrep)
grep --include "*.php" -rlE
'viagra|pharma|Tadacip|eval|base64_decode|socket_shutdown|socket_clos
e|socket_clear_error|fopen|curl' www/
```

En outre, il s'avère parfois très pratique d'obtenir un peu de contexte concernant les résultats, pour cela, nous utiliserons l'option `-C` qui permet de définir le nombre de lignes que l'on veut afficher avant et après l'expression matchée.

Enfin, comme le souligne Erwan dans les commentaires, l'option `-v` fait office de filtre inverse : est affiché tout ce qui ne contient pas le motif.

`file`

Détermine le type d'un fichier indépendamment de son extension. Il suffit de lui fournir en paramètre le fichier à évaluer.

`split`

Coupe un fichier en fichiers plus petits (-l préciser un nombre de lignes, -b préciser une taille en bytes [faites suivre la taille de K, M, G, T pour définir une unité différente]). Pour créer des fichiers de 300 lignes `split -l 300 test.txt`, ou des fichiers de 1MB `split -b 1000000 mon_fichier`.

On reconstitue ensuite le fichier original avec `cat` :

```
# dans un répertoire où il n'y a que les "morceaux"
for f in *; do cat $f >> fichier_original; done

# sinon, comme ils sont préfixés par "x"
# en s'assurant que rien d'autre ne commence par la lettre "x"
for f in x*; do cat $f >> fichier_original; done
```

`sed`

Dans les commandes qui permettent de manipuler du texte, `sed` est sans conteste l'une des plus puissantes. Remplacement selon une expression régulière, effacement de certaines expression où ligne selon un mot-clef donné… Seul inconvénient, qui dit puissance dit aussi complexité. `sed` ne s'explique pas en deux lignes et c'est pour cela que j'y ai consacré un article entier. Ça vaut le coup !

`awk`

`awk` est un langage de programmation à elle seule. Cette commande permet la recherche de chaines et l'exécution d'actions en fonction des motifs trouvés. D'une puissance redoutable, elle est aussi assez complexe. Je vous redirige donc vers des tutos en français ici ou là.

`locate`

Cette commande permet de localiser un fichier sur le disque dur. `locate monfichier.txt`. La commande `locate` est très rapide car elle retrouve le fichier en consultant une base de données. Elle ne parcourt pas directement le disque dur à la recherche du fichier en question. L'inconvénient de ce procédé est que si le fichier est tout récent, il risque de ne pas encore être indexé, et `locate` ne vous sera alors d'aucun secours. On peut forcer la mise à jour de la base de données avec la commande `sudo updatedb`. On peut aussi se tourner vers la commande `find`.

`find`

La commande `find` est bien plus puissante que `locate`, mais elle est aussi bien plus lente car elle parcourt le disque au fur et à mesure de la recherche. Sa syntaxe est la suivante `find /adresse_du_repertoire_de_recherche/ element_a_trouver`. Cette syntaxe n'est qu'une base. `find` permet en effet de rechercher selon une taille ou une date de dernier accès, mais encore d'effectuer des actions sur les fichiers trouvés, d'appeler une commande etc. Une page de man à lire donc…

Connaître la date de dernier accès est très pratique et puissant, quelques exemples :

```
# le plus simple chercher un fichier dont on connait le nom
# trouver tous les .htaccess dans /var/www
find /var/www -name ".htaccess"

# liste les fichiers dans www par date de dernière modification
find /www -type f -printf '%TY-%Tm-%Td %TT %p\n' | sort -r

# on peut être plus précis en affichant les fichiers modifiés dans le
x (ici 50) dernières minutes
find /target_directory -type f -mmin -50

# ou les 24 dernières heures
find /target_directory -type f -mtime -24

# on peut enfin choisir une intervalle (entre hier et avant-hier)
find /target_directory -type f -mtime -48 ! -mtime -24

# last but not least, déplacer les fichiers qui ont plus de 24h (ou
modifié il y a plus de 24h)
find /target_directory -type f -mtime -24
```

Vous noterez très certainement que nous utilisons ici `mtime` pour la dernière **m**odification. Il y a également :

- `mmin` pour les minutes,
- `atime` et `amin` pour le dernier **a**ccès,

- `ctime` et `cmin` pour le dernier **c**hangement.

Vous pouvez lire cet article et ce post pour bien comprendre les différences et implications *change time* et *modification time*.

Flux de redirection

`>`

Renvoyer le résultat dans un fichier (si celui-ci existe, il sera écrasé).

`>>`

Renvoyer le résultat dans un fichier (si celui-ci existe déjà, ajoute le résultat à la fin) il existe deux sortie : 1 la sortie normale, 2 la sortie d'erreurs.

`2>`

Créer un fichier pour les erreurs. `2>>` existe aussi.

`2>&1`

Fusionne les deux sorties dans un seul fichier ex : `./script_de_la_mort.sh > fichier.log 2>&1`.

`<`

Prendre un fichier en entrée. ex : `cat < fichier.txt`.

`<`

Prendre en entrée le clavier au fur et à mesure. Ceci nous permet de passer des données directement à une commande sans avoir besoin de créer de fichier. Mettons que nous voulions trier des prénoms par ordre alphabétique. Nous allons pour cela invoqué la commande `sort`, mais au lieu de créer un fichier texte puis de le faire réorganiser par `sort`, nous allons directement lui soumettre les noms en les entrant au clavier :

```
#on place un mot clef après le "<<" qui sert à délimiter les données
passées en entrée.
#Ce mot clef est tout à fait arbitraire
#on appuie sur "entrer" après chaque prénom pour séparer les
différentes données
sort << STOP
> antoine
> clement
> quentin
> jordan
> mathilde
> clementine
> elena
> helena
> pierre
> STOP
antoine
clement
clementine
elena
helena
```

```
jordan
mathilde
pierre
quentin
```

|

Rediriger le résultat d'une commande dans une autre. ex: `sort prenoms.txt | uniq`.

Multitâche et programme en arrière-plan

`&`

Mettre & à la fin d'une commande permet d'en lancer une autre sans attendre la fin de la première ex : `cp video.avi /users/buzut/desktop/copie-video.avi &`. Cette instruction permet aussi de passer la commande en arrière-plan et redonne immédiatement accès au shell.

`nohup`

Lance le programme et le maintien même une fois la console fermée. Les sorties 1&2 sont redirigées vers `nohup.out`. Exemple :

```
nohup ffmpeg -i video-source.mkv -vcodec libx264 -preset slow video-reencodee.mp4
```

`Ctrl` + `z`

Mettre en pause le processus en cours.

`bg`

Passer le processus qui est en pause en arrière-plan.

`fg`

Reprendre un processus en premier plan (si plusieurs tournent en même temps, `fg %n°`).

`at`

Programme une tache à exécuter à une heure ultérieure ex : `at 18:22` ou `at now + 5hours` puis `ctlr` + `d`.

`atq`

Lister les jobs en attente.

`atrm`

Supprimer des jobs.

`sleep`

Cette commande permet de faire une pause entre l'exécution de deux commandes.
Exemple : `touch gt.txt && sleep 10 && rm gt.txt`
La pause est exprimée en seconde par défaut, il est cependant possible de changer cela en faisant suivre le nombre d'une unité : `m`, `h`, ou `d` pour respectivement les minutes, heures et jours.

`crontab`

crontab est en fait une commande qui permet de lire et de modifier un fichier appelé la "crontab". Voici les options les plus courantes :

- `e` modifie la crontab,
- `l`'affiche la crontab actuelle,
- `r` supprime votre crontab. Attention, la suppression est immédiate et sans confirmation !

`screen`

Multiplexeur de terminaux. Sous ce terme un peu barbare se cache en fait une sorte de terminal virtuel. Vous êtes au boulot, vous ouvrez un terminal et vous le nommez, vous lancez une tache (un script qui va ré encoder plusieurs GB de vidéos par ex, ce qui prend du temps), et vous vous déconnectez, vous arrivez chez vous, vous ré ouvrez votre terminal et retrouvez votre tâche comme si tout était resté ouvert en face de vous.

```
# créer un screen
screen -S nom_du_screen

# pour le détacher (cela veut dire qu'il n'est plus affiché mais
reste actif),
# il faut faire ctrl a puis d

# pour se rattacher à un screen détaché,
screen -r nom_du_screen

# pour lister les screens ouverts
screen -ls

# pour quitter/fermer un screen, comme pour fermer un terminal -->
exit

# il n'est pas possible de s'attacher à un screen non détaché
# (screen non détaché dans un terminal auquel on tente de s'attacher
depuis un autre)
# on peut en revanche partager un screen,
# on voit alors toutes les commandes tapées dans l'un ou l'autre des
terminaux
# pour se connecter à un screen non détaché
screen -x nom_du_screen
```

Vous voudrez parfois remonter dans l'écran du screen, pour cela, il faut passer en *copy mode* avec les touches `ctrl` + `a` puis `[`. Vous pourrez alors naviguer avec les flèches directionnelles. Enfin pour sortir de ce mode `ctrl` + `a` puis `esc`.

Droits, groupes et utilisateurs

`sudo`

Exécuter une commande en tant que root.

`sudo su`

Passer root et le rester.

`chmod`

Changer les droits sur un fichier un ou dossier (option `-R` pour la récursivité dans tous les fichiers et sous-dossier du dossier sur lequel on l'applique).

`chown`

Change le propriétaire d'un fichier/dossier (ne peut s'utiliser qu'en root) option -R pour la récursivité.

`adduser`

Ajouter un utilisateur.

`passwd`

Changer le mot de passe d'un user | ex : `passwd roger`.

`deluser`

Supprimer un user (option `--remove-home` pour supprimer tous ses fichiers).

`addgroup`

Créer un groupe.

`usermod`

Modifie un utilisateur (options : `-l` pour changer le nom, `-g` pour lui assigner un groupe, `-G` pour lui assigner plusieurs groupes (séparés par des virgules), `-a` en complément de `-g` ou `-G`, ajouter des nouveaux groupes au lieu de tout redéfinir) ex, ajouter le groupe video à l'utilisateur buzut, sans supprimer les groupes auxquels il appartenait avant : `usermod -aG video buzut`.

`delgroup`

Supprimer un groupe.

`groups`

Vérifie dans quels groupes est un utilisateur `groups myuser`.

`chgrp`

Change le groupe propriétaire d'un fichier (équivalent à `chown user:group`).

Charge CPU et usage RAM

`free`

Indique l'allocation de la ram et la mémoire libre restante.

`vmstat`

Info ram, swap, cpu.

`/proc/meminfo`

Le fichier `/proc/meminfo` contient de nombreuses informations sur la mémoire. Il suffit d'en afficher la sortir avec `cat /proc/meminfo`.

`tload`

Affiche la charge CPU sous forme de graphique.

`ps -ef`

Afficher tous les processus lancés. Alternativement, on peut utiliser la syntaxe BSD : `ps aux`.

`ps -ejH`

Afficher processus en arbre.

`ps -u`

Lister les processus lancés par un utilisateur donné ex: `ps -u buzut`.

`top`

L'activité du système en temps réel : load, RAM, SWAP processus… `top` a l'avantage d'être installé presque partout.

`htop`

C'est une version améliorée de top, un peu plus graphiques, les infos y sont plus claires et il est possible de trier/ordonner l'affichage selon certains critères.

`glances`

Similaire à `top` et `htop`, `glances` est le tableau de bord de votre machine car il réunit en un coup d'œil toutes les métriques importantes : cpu, load, ram, swap, i/o disques, remplissage des disques. C'est l'œuvre de l'ami Nicolargo et son outil est maintenant intégré aux dépôts des dernières versions Debian & co (je ne sais pas pour les autres distribs).

`atop`

On a parlé ci-dessus de `top` et `htop`, mais il y a aussi `atop` qui est très utile lorsqu'on doit faire un diagnostic un peu plus poussé. De manière générale, j'utilise `htop` en remplacement de top car il est plus ergonomique et lisible, mais quand quelque chose cloche sur le serveur, direction `atop` ! Pour aborder sereinement la bête, je vous conseille la lecture de cet article de linuxpedia, admirable de clarté.

`iotop`

Dans la ligné des `*top`, voici `iotop` qui, comme son nom le laisse entendre, permet d'avoir un aperçu temps réel de l'I/O disque.

Système

`w`

Qui est connecté et fait quoi.

`who`

Qui est connecté.

`date`

Donne l'heure.

`timedatectl`

Dans les versions modernes de Debian et ses dérivés, c'est la commande qui remplace ntpdate pour la synchronisation de l'heure avec un serveur ntp. Par ailleurs, votre serveur aura toujours besoin de `ntp` pour la synchronisation.

Assurez-vous qu'il est bien installé avec un coup de `apt` et contrôlez ensuite l'état en appellant juste la commande `timedatectl` sans arguments. Voici les commandes les plus utiles :

```
# activer/désactiver la synchro auto
timedatectl set-ntp true/false

# lister les fuseaus horaires
timedatectl list-timezones

# changer de fuseau horaire
timedatectl set-timezone Europe/Paris

# régler l'heure manuellement
timedatectl set-time 'HH:mm:ss'

# régler la date
timedatectl set-time 'A:M:J'
```

Vous pouvez par ailleurs spécifier les serveurs NTP avec lesquels vous voulez synchroniser depuis le fichier de config `/etc/systemd/timesyncd.conf`. Pensez aussi que le port NTP (123 en UDP) doit être ouvert sur votre machine.

`uptime`

Temps depuis mise en route + charge (charge moyenne 1 - 5 - 15 mn).

`swapoff`

`swapoff -a` permet de désactiver le(s) swap tandis que son pendant `swapon -a` permet d'activer le(s) swap. L'exécution de `swapoff -a && swapon -a` permet donc de forcer la purge du swap.

`kill`

Tuer un processus (va demander son PID). L'option `-9` force à quitter. Comme le souligne Httqm dans les commentaires, il s'agit du dernier recours car les ressources utilisées par le processus ne seront pas libérées. Comme le fait remarquer Kelec en commentaires également, il est possible de libérer la RAM avec `sysctl` (voir un peu plus bas).

`killall`

Quitte toutes les occurrences d'un programme.

`reboot`

Redémarrer le système d'exploitation.

`shutdown`

Programmer un redémarrage ou un arret.

`poweroff`

Bien qu'assez similaire à shutdown dans la mesure où elle permet d'éteindre le système, poweroff permet aussi selon les arguments qui lui sont passés, de rebooter ou de changer de runlevel. Vous pouvez aussi consulter ce thread [en] qui explique les différences entre `poweroff` et `shutdown`.

`halt`

Permet "l'arrêt" du système. Je mets *arrêt* entre guillemets car le système peut rester sous tension avec cette commande (selon les options passés et les paramètres par défaut du système). Regardez les différences entre shutdown et halt [en].

`last`

Historique des connexions.

`lsof`

list open files, dresse la liste des fichiers ouverts. Comme le fait remarquer l'ami MagiCrazy dans les commentaires, cette commande peut s'avérer bien utile pour voir quel fichier bloque le démontage d'un filesystem par exemple.

`hostname`

Affiche le nom d'hôte de la machine conformément à ce qui est écrit dans le fichier `/etc/hostname`.

`uname`

Infos sur le système et le matos. L'option `-r` bien pratique, permet d'obtenir la version du kernel.

`lsb_release`

`lsb_release -a` donne toutes les infos sur la distrib.

`lshw`

Donne une liste détaillée de l'hardware système tels que la configuration ram, la version du firmware, la configuration de la carte mère… Avec l'option `-short` vous obtiendrez une sortie plus digeste. L'option `-c network` s'avère aussi bien utile pour connaître le nom d'une interface réseau encore non configurée avec le standard de nommage systemD [en].

`lsblk`

Liste tous les devices de type bloc (disque dur).

`lspci`

Liste tous les périphériques PCI.

`lsusb`

Liste tous les périphériques USB.

`/proc/version`

Fichier qui contient des infos sur le noyau. On peut afficher son contenu avec `cat` : `cat /proc/version`.

`/proc/cpuinfo`

Fichier qui contient des infos sur le processeur. On peut afficher son contenu avec `cat` : `cat /proc/cpuinfo`.

`sysctl`

Cette commande permet d'afficher et de configurer les paramètres du noyau (à chaud). Très pratique avec un coup de `grep`, `sysctl -a` vous donnera tous les paramètres du noyau. L'option `-p` est également intéressante car elle permet de recharger les paramètres soit depuis `/etc/sysctl.conf` par défaut, soit depuis un fichier passé en paramètre. Dans les choses pouvant s'évérer très utiles, `sysctl -w vm.drop_caches=3` permet de libérer le cache de la RAM.

`dmidecode`

Lit les infos du bios.

`dmesg`

Affiche les messages du buffer du noyau.

`service`

Cette commande permet de gérer les services. Lancer, arrêter, lister les services du système etc. Par exemple, pour relancer nginx après un changement dans le fichier de config, on fera `service nginx restart` (notez que `reload` suffit dans certains cas). La commande `service --status-all`, bien pratique, permet de lister tous les services disponibles sur le système.

Le paramètre `status` s'avère souvent fort utile puisqu'il donne des informations sur un service en particulier, notamment s'il est actif ou non. Exemple `service nginx status`.

On peut également activer ou désactiver le démarrage automatique des services au boot, pour cela, on doit troquer la commande `service` pour la commande native `systemctl`. Prenons apache2 pour exemple :

```
# activer/désactiver un service
systemctl enable apache2
systemctl disable apache2

# vérifier si un service est lancé au boot
systemctl is-enabled apache2
```

`journalctl`

Cette commande, qui accompagne systemd, permet de visionner et requêter les journaux système. Nous ne pourrons bien entendu pas aborder toutes les options ici, cependant, à vous d'aller fouiller le web pour répondre à vos cas d'usage (requête sur un horaire donnée, process etc). La commande `journalctl` seule, affiche l'ensemble des logs que `journald` a collecté.

Pour ma part, un exemple bien pratique concerne les boots. Que s'est-il passé durant le dernier boot, ou le précédent ?

```
# on commence par lister les différents boots présents dans les logs
# (il se peut qu'il n'y en ait qu'un)
journalctl --list-boots
 0 3774424e8f5a45cb930b1eef3b8355dd Tue 2017-08-15 14:45:48 CEST—Wed
2017-08-16 12:54:57 CEST

# on demande ensuite à afficher les logs correspondants à ce boot
journalctl -b -0
- Logs begin at Tue 2017-08-15 14:45:48 CEST, end at Wed 2017-08-16
12:56:23 CEST. --
[…]
```

`make`

Permet de compiler un programme dont on détient les sources. En général on fait tout d'abord. /`configure` [à lancer avec `--help` pour voir les différentes options de compilation] (lance le script de configuration qui vérifie la présence de toutes les

dépendances, et écrit le fichier makefile qui contient les ordres de compilation), `make`, et enfin `make install` (elle installe le logiciel).

`update-rc.d`

`update-rc.d`permet de configurer le démarrage ou l'arrêt automatique de service au démarrage de la machine ou selon le runlevel. On donne en argument le nom du service et l'action (remove ou default pour l'ajout) `update-rc.d -f apache2 remove`; `-f` permet de forcer l'effacement du lien symbolique même si le nom existe encore. On peut aussi placer un script de démarrage dans répertoire `/etc/init.d` ou le renseigner le fichier `/etc/rc.local` (qui a lui-même un lien symbolique dans /etc/init.d).

`/etc/passwd`

Bien qu'il ne s'agisse pas, techniquement, d'une commande, il me semble important de connaître la structure du fichier `/etc/passwd`. Lequel regroupe l'ensemble des utilisateurs du système et de leurs informations.

```
# exemple pour l'utilisateur sensu
# 1  :2: 3 : 4 :            5             :     6    :     7
sensu:x:999:999:Sensu Monitoring Framework:/opt/sensu:/bin/false
```

1. Nom de l'utilisateur,
2. mot de passe (x signifie que le mdp est chiffré dans le fichier `/etc/shadow`,
3. l'id de l'utilisateur (0 est pour root et les id de 1 à 99 sont réservés pour les comptes prédéfinis),
4. l'id du groupe tel que défini dans `/etc/group`,
5. champ de commentaire,
6. répertoire "home" de l'utilisateur,
7. le shell par défaut (`/bin/false` et `/usr/sbin/nologin` signifient que l'utilisateur n'a pas de shell).

`/etc/group`

Fichier qui contient les groupes utilisateurs de la machine (ce qui inclut les groupes utilisés par les logiciels ex : www-data pour Apache). On peut afficher les informations avec `cat` : `cat /etc/group`.

`which`

Localiser une commande ex :

```
which cat
/bin/cat
```

`whereis`

Localiser un fichier binaire.

Gestionnaire de paquets Debian et dérivés

`apt-cache`

> Gestion des paquets. Deux options sont très utiles `apt-cache search nom_paquet`, permet de chercher un paquet, et `apt-cache show`, permet d'obtenir des détails sur un paquet.

`apt-get`

> Gestion des paquets. Les commandes que l'on utilisera le plus sont `update` (MAJ des sources de paquets dispos), `upgrade` (mise à jour du système et autres softs), `install` (`apt-get intall truc-à-installer` pour installer un nouveau logiciel et ses dépendances), `purge` (permet de désinstaller un paquet de manière plus "propre" que remove car cela efface aussi les fichiers de configuration).

`apt`

> Apparue assez récemment, `apt` est décrit dans son man comme le front-end utilisateurs pour un usage plus interactif d'autres outils spécialisés tels que `apt-get` ou `apt-cache`. Il offre dans l'ensemble les mêmes possibilités que `apt-get`. Je retiens une commande toute particulière : `apt list --upgradable` qui permet de lister les packets qui seront mis à jour si l'ont fait un `upgrade` (avec `apt` ou `apt-get`).

`aptitude`

> C’est un autre utilitaire de paquets. Plus récent qu'apt-get, il est installé en parallèle de celui-ci sur Ubuntu et Debian. Préférez-le à apt-get. Il s'utilise dans l'ensemble comme apt-get mais est plus performant.

`add-apt-repository`

> Permet d'ajouter des dépôts alternatifs aux dépôts officiels. C'est très utile car les dépôts officiels ont souvent du retard sur les versions de logiciels que sortent les développeurs et certains logiciels en sont même absents. Ainsi, en ajoutant par exemple les dépôts des développeurs, vous pouvez bénéficier des dernières versions juste en vous servant de `apt-get` ou `aptitude`, sans avoir besoin de compiler !
>
> Par exemple, FFMPEG avait été supprimé des dépôts officiels d'Ubuntu (réintégré en version 15.04) au profit de Libav, son fork. Pour profiter des dernières versions de FFMPEG sans avoir à compiler manuellement à chaque fois :

```
add-apt-repository ppa:kirillshkrogalev/ffmpeg-next

# on met notre liste de packets à jour
apt-get update

# si ffmpeg n'est pas installé ont l'installe
apt-get install ffmpeg

# s'il est déjà installé,
# un upgrade se chargera de le mettre à jour
```

```
apt-get upgrade
```

Il est également parfois pratique de pouvoir supprimer de vieux dépôts, pour cela, il suffit de faire appel à l'option `-r`.

Dernière astuce, ce n'est pas une commande spécifique, mais une combinaison de commandes qui permettent de rechercher un dépôt sur le système :

```
grep ^ /etc/apt/sources.list /etc/apt/sources.list.d/* | grep
nom_du_ppa
```

`apt-key`

Cette commande va bien souvent de pair avec l'ajout de dépôts puisqu'elle permet de gérer les clefs cryptographiques en validant l'authenticité. On utilisera le plus souvent `apt-key adv` pour ajouter de nouvelles clefs, Ubuntu-fr en détaille très bien l'usage. `apt-key list` permet de lister toutes les clefs installées et `apt-key del` permet d'effacer une clef.

`apt-cache madison`

Autre commande bien pratique, `apt-cache madison nom_du_packet` affiche les différents dépôts liés à un packet donné et les versions actuelle de chacun d'entre eux.

`dpkg`

Info sur les paquets installés (options pour lister tous les paquets, désinstaller etc) ex : liste des paquets installés `dpkg --get-selections`. L'option `-l` fourni également une liste exhaustive et avec une petite description de chaque packet, ce qui peut s'avérer très pratique. On constate parfois qu'un grand nombre de paquets sont marqués pour être désinstallés avec le tag `deinstall`, pour tout enlever d'un coup :

```
dpkg --purge `dpkg --get-selections | grep deinstall | cut -f1`
```

update kernel

Si votre partition `/boot` est indépendante et qu'elle n'est pas très grande, il est probable qu'après un certain temps, vous deviez faire un peu de ménage, sans quoi l'espace nécessaire à une **mise à jour du kernel** est insuffisant. On devra donc supprimer les anciens noyaux, pour ce faire,

```
apt-get purge $(dpkg -l linux-{image,headers}-"[0-9]*" | awk
'/ii/{print $2}' | grep -ve "$(uname -r | sed -r 's/-[a-z]+//')")
```

Il ne vous reste alors plus que le kernel actuellement utilisé et vous pouvez maintenant effectuer votre mise à jour sans problème. Vous trouverez plus de détails sur cette commande sur ce thread askubuntu.

Disques et volumes

`df`

Remplissage des disques (l'option `-h` permet d'obtenir les tailles en "human readable").

`mount`

Permet de monter le prériphérique d'un système de fichier sous un répertoire local. Par exemple, pour monter la partition `/dev/sdb1` au point de montage `/home` : `mount /dev/sdb1 /home/`. Inversement, nous utilisons `umount` pour démonter le volume.

`fdisk`

Permet de gérer les partitions. Affiche la table des partitions si on utilise l'option `-l`. Pour créer ou modifier des partitions, on utilisera le mode interactif `fdisk /dev/sdx` ou `/dev/sdx` est le chemin vers votre disque. Ensuite, l'option `m` listera toutes les commandes possibles, laissez-vous guider ! Vous pouvez jetez un œil à l'article sur le partitionnement Linux pour savoir quels types de partitions créer et pourquoi.

`parted`

Semblable à `fdisk`, `parted` supporte les partition GPT. L'option `-l` permet d'afficher la table des partitions.

`fsck`

Cet utilitaire permet de vérifier et de réparer le système de fichier.

`dd`

L'outil permet d'effectuer des opérations sur les disques, notamment de les effacer, par exemple `dd if=/dev/zero of=/dev/sdc` ou de cloner un disque ; ex : `dd if=/dev/sda1 of=/dev/sdb1 bs=64K conv=noerror,sync`.

`ddrescue`

Lorsqu'on fait face à un disque dur endommagé, il vaut mieux tenter de le cloner avec `ddrescue` qu'avec `dd`. Le premier est en effet dédié à cet usage. Il peut effectuer une première passe pour récupérer le maximum de données en ignorant les secteurs endommagés, puis dans une seconde passe il tentera de récupéré les données endommagées. La doc d'Archlinux détaille bien le processus de recovery.

`hdparm`

La commande `hdparm` permet de récupérer des infos, de tester et de modifier la configuration – bas niveau – des disques dur. On peut par exemple modifier la taille des buffers avant écriture, activer ou désactiver ou modifier l'économie d'énergie, modifier la vitesse de mouvement des têtes de lecture… bref beaucoup de choses. On peut aussi irrémédiablement endommager le disque, prudence donc.

Dans les options bien utiles, voici ce que je retiens :

- `hdparm -I` affiche toutes les infos sur le disque (type de disque, taille des caches, vitesse de rotation, options supportées…).
- `hdparm -tT /dev/sda` fournit la vitesse de lecture pour les contenus avec et sans cache (à répéter plusieurs fois avec le serveur idle pour des résultats pertinents).

Enfin, hdparm s'avère parfois salvateur en cas de secteurs défectueux.

```
# si vous avez ce genre de messages dans votre syslog
Add. Sense: Unrecovered read error - auto reallocate failed
sd 3:0:0:0: [sdd] CDB:
Read(10): 28 00 e5 63 34 18 00 00 18 00
blk_update_request: 46 callbacks suppressed
end_request: I/O error, dev sdd, sector 3848483864

# on peut faire un check de l'état du secteur
hdparm –read-sector 3848483864 /dev/sda
/dev/sda: Input/Output error

# il est bien défectueux, on force donc sa réallocation en écrivant
sur ce secteur
hdparm –write-sector 3848483864 /dev/sda

/dev/sda:
Use of –write-sector is VERY DANGEROUS.
You are trying to deliberately overwrite a low-level sector on the
media
This is a BAD idea, and can easily result in total data loss.
Please supply the –yes-i-know-what-i-am-doing flag if you really want
this.

Program aborted.

# il faut bien mettre le paramètre –yes-i-know-what-i-am-doing
# sans quoi hdparm refuse d'accéder à notre requête
$ hdparm –write-sector 3848483864 –yes-i-know-what-i-am-doing
/dev/sda

/dev/sdb: re-writing sector 3848483864: succeeded

# on retente une lecture du secteur
hdparm –read-sector 3848483864 /dev/sda
/dev/sda:
reading sector 3848483864: succeeded
```

`smartctl`

`smartctl` permet d'afficher les informations smart d'un disque. `smartctl -a /dev/sda` affiche toutes les informations à propos de sda. La fiche Wikipedia à propos du SMART fournit de bonnes explications sur les différentes données et leur interprétation.

`/proc/mdsat/`

Fichier qui contient les infos sur vos RAID logiciels. On peut afficher ces informations en faisant un `cat` : `cat /proc/mdstat`. Lisez l'article sur les différents

niveaux de RAID (simulateur intégré) pour trouver le RAID qui vous correspond Vous pouvez également consulter ce wiki [en] pour vous orienter dans la jungle des infos délivrées par `mdadm`..

`mdadm`

Commande qui permet d'obtenir des infos sur les RAID soft et de les paramétrer (sortir un disque de l'array, en rajouter un, le reconstruire en cas de disque défaillant…). Par exemple `mdadm --detail /dev/md0` donnera tous les détails lié à un l'array. Notez que cette commande est assez proche de l'option `--examine`, mais laquelle s'applique à un disque constitutif d'un raid `--examine /dev/sd*` et non au volume raid. Cette commande étant très riche et servant de nombreuses fonctions, je vous laisse vous référer au man pour plus de détails. On peut aussi trouver de précieuses informations sur le linux raid wiki.

Web

Web veut tout dire et rien dire à la fois, surtout dans la mesure où on parle déjà d'un serveur. Cependant, cette section concerne tout particulièrement les commandes liées aux serveurs web. On pense évidemment immédiatement à Apache2 mais d'autres pourraient s'y rajouter.

`a2ensite`

Activer un `vhost` Apache : `a2ensite buzut`

`a2dissite`

Désactiver un `vhost` Apache : `a2dissite buzut`

`a2enmod`

Activer un module Apache : `a2enmod rewrite`

`a2dismod`

Désactiver un module Apache : `a2dismod rewrite`

`a2enconf`

Sur le même modèle que pour les vhost et les modules, `a2enconf` et `a2disconf` permettent d'activer ou de désactiver des configurations. Par exemple, lorsqu'on ne renseigne pas explicitement le fichier d'`access.log` d'un vhost, Apache log le tout dans un fichier dédié : `other_vhosts_access.log`. Dans le cas où l'on ne voudrait pas de logs d'accès par exemple, on peut désactiver cette configuration : `a2disconf other-vhosts-access-log`.

`apache2ctl`

`apache2ctl` permet d'une part d'agir en tant que script init (ce qui n'a pas grand intérêt puisqu'on utilise en général la commande `service`). Et d'autre part de controller le processus Apache et de récupérer des infos sur ce dernier. `apache2ctl -S` est assez utile puisqu'elle permet de voir comment apache interprète nos Vhost, l'option `-M` liste les modules Apache activés et `status` affiche un résumé de l'état actuel du serveur (connexions, nombre the workers…). Enfin, bien pratique, l'option `-t` ou `configtest` vous permet de vous assurer que vos fichiers de config n'ont pas d'erreur de syntaxe.

Gestion réseau

`ssh`

secure shell. Permet de se connecter au shell d'un ordinateur distant et d'y exécuter des commandes.

```
ssh login@ip ou nom_hôte
```

Et voilà tout, pour se déconnecter, ce sera exit, et toutes les commandes habituellement utilisables dans un terminal le sont aussi via ssh. Une option bien pratique de ssh est le tunneling qui permet par exemple de déjouer les pare-feux par la mise en place d'un proxy socks. Si certains ports sont bloqués et vous empêchent de vous servir de tel ou tel service ou d'accéder à tel ou tel site (en vacances en Chine ?), la solution est donc de tout rediriger vers un port local et de laisser la machine distante – qui n'est pas derrière un part feu – accéder aux ressources non autorisées. Nous allons donc rediriger tout notre trafic vers un port prédéfini (en l'occurrence 2013), à travers notre connexion SSH.

```
ssh -D 2013 login@ip_serveur_distant
```

Dernière étape pour que cela fonctionne, il faut que votre trafic sortant soit redirigé vers le port 2013. Vous pouvez le configurer dans votre navigateur et pour l'ensemble des connexions de votre ordinateur dans les préférences réseau.

`fail2ban`

Il est difficile de ne pas évoquer fail2ban lorsque l'on parle de ssh. Ce petit logiciel scrute les logs de votre serveur et bloque les ips qui semblent malveillantes. Comme la configuration est un peu plus longue qu'un simple listing d'une ou deux commandes, j'y dédis un article entier.

`ifconfig`

Cette commande des plus indispensables permet d'obtenir des infos et de configurer les interfaces réseau. Employée sans argument, elle fournit des infos sur les interfaces réseaux. Mais elle permet aussi de modifier la configuration. Par exemple pour changer une adresse mac, on utilisera la commande `ifconfig $INTERFACE ether $MAC`. Donc pour changer l'adresse mac de la carte ethernet, ce sera en général `ifconfig eth0 ether 5E:FF:56:A2:AF:15`. Notez cependant que cette commande, bien que toujours fonctionnelle, est dépréciée en faveur de la commande `ip` ci-dessous.

`ip`

Cette commande permet d'afficher et de manipuler le routage et les interfaces. On s'en sert souvent pour lier ou supprimer une ip à une interface :

```
# lister toutes les addresses
ip addr

# Ne lister que les informations ipv6
ip -6 addr
```

```
# lister une interface en particulier
ip addr show dev em2

# ajouter une adresse ipv4
ip addr add 192.168.0.7 dev eth0

# supprimer une ipv4
ip addr del 192.168.0.7 dev eth0

# pour la v6, il faut utiliser l'argument -6
ip -6 addr del 2574:104::b08b:c107 dev eth0
```

`ping`

Permet de pinguer un client pour voir s'il est en ligne ou s'il répond au ping, ex `ping google.fr`. L'option `-c` permet de préciser le nombre de ping à envoyer avant que la commande ne s'arrête (elle prend donc en argument un nombre entier exemple : `ping -c 8 google.fr`), l'option `-f` permet de flooder, c'est à dire que la carte réseau enverra autant de ping qu'elle est capable d'envoyer par seconde.

`traceroute`

Trace la route d'un paquet, routeur par routeur, jusqu'à sa destination. On peut utiliser indifféremment une ip ou un nom de domaine.

```
traceroute to google.com (173.194.67.113), 64 hops max, 52 byte
packets
 1  fast3504 (192.168.1.254)  12.108 ms  1.239 ms  1.217 ms
 2  sl869-h01-31-38-37-254.dsl.sta.abo.bbox.fr (42.58.37.254)  33.504
ms  32.771 ms  40.120 ms
 3  173.la63.bsr01-lyo.net.bbox.fr (194.158.110.189)  32.684 ms
31.828 ms  31.238 ms
 4  be19.cbr01-cro.net.bbox.fr (212.194.171.16)  41.761 ms  41.636 ms
40.140 ms
 5  be1.cbr01-ntr.net.bbox.fr (212.194.171.1)  45.495 ms  42.217 ms
40.052 ms
 6  * * *
```

`dig ndd`

vérifier une correspondance dns. On peut directement spécifier l'ip du serveur dns à interroger, par exemple, `dig @8.8.8.8 buzut.fr`.

```
Buzut:~ Buzut$ dig @8.8.8.8 buzut.fr

; <<>> DiG 9.6-ESV-R4-P3 <<>> @8.8.8.8 buzut.fr
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 2660
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;buzut.fr.                      IN      A

;; ANSWER SECTION:
buzut.fr.         43200   IN      A       213.186.33.4

;; Query time: 155 msec
```

```
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Fri Jul 27 09:01:06 2012
;; MSG SIZE  rcvd: 42
```

`host`

Permet de vérifier le reverse. Vous entrez donc une adresse ip et la commande vous retourne le nom de domaine associé.

`nslookup`

Un peu dans le même genre que dig, `nslookup` fourni des infos sur le nom de domaine passé en argument, adresse ip, type de réponse DSN… Exemple : `nslookup buzut.fr`.

```
Buzut:~ Buzut$ nslookup buzut.fr
Server:          212.27.40.240
Address: 212.27.40.240#53

Non-authoritative answer:
Name:    buzut.fr
Address: 213.186.33.4
```

`Route`

Cette commande d'une praticité sans pareil vous permet de visionner les routes, mais aussi, en lui spécifiant quelques arguments, de modifier les routes. Pour ajouter une route par défaut :

`route add default addr ip` et pour supprimer une route par défaut : `route delete default`.

`arp -an`

Cette commande permet de voir la table arp actuelle. C'est à dire la correspondance entre les adresses ip et mac sur votre réseau.

`wget`

Télécharge un fichier distant sur l'ordinateur. `wget http://www.site.org/rep/01/fichier.txt`.

`curl`

`curl` permet, comme `wget`, de récupérer un fichier depuis une url, mais s'il mérite sa place ici, c'est qu'il permet bien plus. C'est en effet le couteau suisse du HTTP. On l'utilisera en effet pour tester des requêtes dans différents formats, analyser les `HEADERS` etc.

```
# requête get classique
curl https://buzut.fr

# afficher les headers
```

```
curl https://buzut.fr -D -

# faire une requête de type X (HEAD, POST, PUT, PATCH, DELETE…)
curl -X HEAD https://buzut.fr

# passer des paramètres au format form data
curl -X POST --data "email=moi@mail.com&passwd=azerty"
https://monsite.com/login

# même requête avec les paramètres en request payload, JSON. (on
ajout ici un header)
curl -H "Content-Type: application/json" -X POST --data
'{"email":"moi@mail.com","passwd":"azerty"}'
https://monsite.com/login
```

`openssl`

Cette bibliothèque est le couteau suisse du chiffrement. Elle permet principalement de travailler avec les certificats SSL/TLS. Voyons quelques-une des options les plus courantes.

```
# générer un mot de passe (format b64 et longueur 32 signes)
openssl rand -base64 32

# générer un certificat ssl auto-signé
openssl req -newkey rsa:4096 -new -x509 -days 365 -nodes -out
cert.crt -keyout cert.key

# Très utile, vérifier ce que contient une clef, (domaines protégés
etc)
openssl x509 -text -noout -in privateKey.key
```

Pour plus de commandes openssl, vous pouvez lire cet article ou jeter un œil à cheat.sh

`scp`

vise à remplir la même fonction que la commande de copie `cp`, mais elle permet de copier les fichiers de manière sécurisé à travers le réseau; c'est à dire entre hôtes distants. De même qu'avec `cp`, l'option `-r` permet de copier un répertoire entier. `scp ficher_exemple login@ip_ou_adresse:adresse_de_destination`.

```
scp test.txt buzut@192.168.0.128:~/transfert

#et pour récupérer un fichier d'un hôte distant
scp buzut@192.168.0.128:~/movie.mkv ~/
```

`nmap`

`nmap` est un outil qui scanne le réseau et les ports réseau d'une machine afin de voir lesquels sont ouverts et de détecter d'éventuelles failles sur les machines. Il permet aussi de détecter des machines connectées au réseau et bien plus encore. J'ai fait un petit billet sur nmap, qui reste un outil de référence dans le monde de la sécurité informatique.

`iftop`

Dans la même veine que top, iftop sert à surveiller toutes les connexions réseau. Attention, iftop nécessite les privilèges root pour être lancé. Si vous n'êtes pas root, pensez à le faire précéder de sudo.

`speedometer`

Un peu plus graphique que `iftop`, `speedometer` monitor le traffic de vos entrées/sorties, permet de surveiller la progression d'un téléchargement, de savoir combien de temps il faudra pour transférer tel fichier ou encore de connaître la vitesse d'écriture de votre système.

`exim4`

Exim est un MTA qui permet d'envoyer des emails depuis le serveur. Sans lui (ou un autre MTA) la fonction `mail()` de php ou d'autres langages ne sera pas effective. Il en existe d'autres mais celui-ci est robuste, sécurisé, modulable et demande peu de ressources. Son installation est très simple, je l'explique dans cet article sur le logging.

Netstat

Ça c'est un morceau, et en faire le tour prendrait (au moins) un article entier. Quoi qu'il en soit cette commande est très pratique pour avoir un aperçu de ce qui se passe sur le réseau. On va donc voir directement quelques combinaisons de paramètres qui reviennent souvent.

`-nr`

Pour la table de routage (revient au même que route).

`-i`

Donne des statistiques sur les différentes interfaces réseau.

`-s`

Personnellement je ne m'en sers que très rarement, mais c'est un résumé de toutes les stats réseaux, alors ça peut être utile de temps à autre.

`-uta`

Liste toutes les connexions ouvertes (`u` pour udp `t` pour tcp et `a` pour afficher toutes les connexions (all)), à vous de jouer avec les paramètres pour filtrer un peu tout ça !

`-nap`

Alternative à `-uta` (plus facile à retenir pour sa signification anglosaxone), cette commande donne un résultat que je trouve plus lisible. Elle affiche toutes les connexions (`a`) avec les adresses en format numérique (`n`) et donne le PID correspondant (`p`).

`-peanuts`

Dans le genre facile à retenir, on a là la combinaison des bons arguments ! Elle donnera un résultat un peu plus détaillé que `-nap` puisqu'elle ajoute UDP et TCP avec du détail (`-e` pour extended) et des statistiques (`-s`) pour toutes les connexions.

`-ltupn`

Encore une variante qui nous permet de visualiser tous les services qui sont à l'écoute sur des ports (donc toutes les connexions ouvertes). Toutes les connexions tcp `-t` et udp `-u`, à l'écoute `-l` avec les programmes correspondant `-p` et accessoirement on demande de ne pas résoudre les noms de protocole avec `-n`.

`-l`

Permet d'afficher toutes les connexions en écoute (listen), on aura donc `netstat -lt` pour toutes les conexions tcp en écoute.

`-n`

Peut-être pratique aussi car elle permet d'afficher les adresses en format numérique au lieu de tenter de déterminer le nom symbolique d'hôte, de port ou d'utilisateur.

`-p`

Permet d'affiche le nom et le PID des processus propriétaires des connexions.

`netstat`

Est la commande historique pour lister les connexions. Cependant, elle a été dépréciée en faveur de [`ss`](https://www.it-connect.fr/les-connexions-sockets-avec-la-commande-ss/) qui est plus moderne et plus performante, surtout en présence de nombreuses connexions. La plupart des options sont les mêmes, vous ne serez donc pas perdu. Au besoin, un petit coup de `man ss` ne peut pas faire de mal.